Technical Requirements

Tech Stack (Pick Your Strength)

- **Frontend**: React OR Angular
- **Styling**: Tailwind CSS OR Material-UI OR Bootstrap (fastest setup)
- **Backend**: NestJS API (**NO** Mock Server), any other backend comfortable and matching to skills claimed
- **Database**: PostgreSQL/MySQL/Any other RDBMS, can pick MongoBD equivalent if justifiable
- **State Management**: Redux Toolkit OR equivalent

Quality Gates (Non-negotiable)

0. **AI Prompts** Complete trace of prompts used
1. **GitHub Repository** with clear README
2. **Local Setup Instructions** (max 3 commands to run)
3. **Responsive Design** (mobile + desktop)
4. **At least 3 unit tests** for core functions
5. **Clean Code** with proper component structure

Assessment Criteria

1. Technical Decision Making

- Feature prioritization rationale
- Tech stack choices and justification
- AI tool selection and usage strategy
- Time management and scope decisions

Deliverable: Brief justification document - Which MVP features you chose and why - Which features you skipped and why - Tech stack decisions and trade-offs - AI tools used and their effectiveness

2. AI Tool Effectiveness

- Strategic use of AI for rapid development
- Quality of AI-generated code integration
- Ability to debug and modify AI outputs
- Documentation of AI-assisted workflow

Evidence Required: - Comments in code showing AI-generated sections - Brief log of AI interactions and modifications made - Demonstration that you understand every line of code

3. Code Quality & Architecture

- Clean, readable component structure

- Proper error handling
- Responsive design implementation
- Basic testing coverage
- Git commit history showing progression

4. Execution & Completeness

- Working functionality as demonstrated
- Professional GitHub repository presentation
- Clear setup and running instructions
- Realistic feature scope for time constraint

Rapid Development Strategy

Recommended AI Tool Workflow:

1. **Project Setup**: Use AI to generate boilerplate
2. **Component Generation**: AI-generate base components, then customize
3. **API/Logic Creation**: AI-assist with CRUD operations
4. **Styling & Responsive**: AI-help with CSS/Tailwind
5. **Testing Setup**: AI-generate basic tests

Smart Shortcuts for 2-Hour Development:

- Use AI to generate initial project structure
- Leverage component libraries (Material-UI, Chakra UI, etc.)
- Implement persistence later fro the get go
- Focus on functionality over visual polish
- Use AI for boilerplate tests, customize the critical ones

Submission Requirements

GitHub Repository Must Include:

1. **Complete source code** with clear folder structure
2. **README.md** with:
    - Feature selection justification (2-3 sentences each)
    - Setup instructions (max 5 steps)
    - Screenshots of working application
    - AI tools used and how
    - What you would add next with more time
3. **package.json** with all dependencies
4. **Clean commit history** showing development progression

Demo Requirements:

- Application runs locally without errors

- Core ticket operations work end-to-end
- Responsive design demonstrated
- Basic error handling in place

Success Metrics

Minimum Passing Criteria:

- Application runs locally in under 5 minutes setup
- Can create, view, and update tickets
- Basic authentication/user context
- Responsive design works on mobile
- Clean, documented GitHub repository
- Reasonable feature selection justification

Excellence Indicators:

- Clean component architecture
- Strategic AI tool usage with clear understanding
- Good UX despite time constraints
- Thoughtful trade-off decisions
- Professional git history and documentation

Success Tip: This is about demonstrating your ability to comprehend the problem statement, analysis, solution approach, coding skills, standards and process adherence. All along taking help from available tools and technologies including AI while making smart technical decisions. Focus on working functionality, but ensure you can explain every choice you made.